rs1Wire: An Efficient Rust Ecosystem for 1-Wire Thermal Sensors Using Linux

*Note: Sub-titles are not captured for https://ieeexplore.ieee.org and should not be used

1st David Costa
School of Computing and Data Science
Wentworth Institute of Technology
Boston, Massachusetts, United States
costad3@wit.edu

2nd Ashar Neyaz
School of Computing and Data Science
Wentworth Institute of Technology
Boston, Massachusetts, United States
neyaza@wit.edu

***Abstract*—This paper is a Work-in-Progress, and as such contains many bulleted lists for my own organizational purposes. The Introduction, and Methodology sections are have been written with the flow of the final paper. Other sections are being converted from bulleted notes to full prose.**

***Index Terms*—Rust, 1-Wire, Linux, Embedded Systems**

I. Introduction

The 1-Wire protocol, developed by Maxim Integrated (now Analog Devices), connects multiple devices over a single data line [1], which makes it well suited to applications where space and wiring are limited. One of the most popular 1-Wire devices is the DS18B20 digital temperature sensor [2], widely used in hobbyist and laboratory settings, particularly with single-board computers (SBCs) such as the Raspberry Pi and BeagleBone. On these boards, Linux provides a 1-Wire subsystem: the ‘w1-gpio‘ bus master drives the bus, and the ‘w1_therm‘ slave driver exposes each sensor’s readings, in millidegrees Celsius, as sysfs files under ‘/sys/bus/w1/devices/‘ [3]. At the same time, Rust is gaining traction in embedded and systems programming for its memory safety and efficient runtime [4]. Yet no maintained Rust library makes full use of the kernel’s 1-Wire interface, and this work aims to fill that gap in both existing software and the literature.

A. Abbreviations and Acronyms

This table is for self use to verify that abbreviations get defined when they are first used.

TABLE I
Abbreviations and Acronyms

Abbreviation/Acronym	Definition
SBC	Single Board Computer
GIL	Global Interpreter Lock
ABI	Application Binary Interface
API	Application Programming Interface
CRC	Cyclic Redundancy Check
EEPROM	Electrically Erasable Programmable Read-Only Memory
FFI	Foreign Function Interface
GPIO	General Purpose Input/Output
HAL	Hardware Abstraction Layer
I2C	Inter-Integrated Circuit
I/O	Input/Output
IRQ	Interrupt Request
MQTT	Message Queuing Telemetry Transport
OS	Operating System
PCIe	Peripheral Component Interconnect Express
PEP	Python Enhancement Proposal
PR	Pull Request
PyPI	Python Package Index
RAM	Random Access Memory
ROM	Read-Only Memory
RT	Real-Time
SoC	System on a Chip
sysfs	System Filesystem
VCC	Positive supply voltage

B. Problem

Rust users on Linux SBCs

Current Rust 1-Wire crates focus almost entirely on implementing the protocol in userspace using bit-banging and the widely utilized embedded-hal crate [5], [6]. This is because many Rust users are embedded developers who work on bare-metal microcontrollers [7] without an operating system. This method is unsuitable for Linux developers because userspace bit-banging is exposed to scheduler preemption and jitter, which can corrupt the timing-sensitive 1-Wire protocol [8]. The only Rust crate that reads kernel output, ‘w1_therm_reader‘, is a small single-sensor parser that its author describes as incomplete: “If you want a complete OneWire library, this crate is not the one you’re looking for” [9]. It lacks features such as device discovery, bulk reading, and configuration, which this project aims to provide. It also panics on malformed input, which is not ideal for robust applications.

Python users

While Rust is in itself a systems programming language and can be used with an operating system, when one is present (e.g., Linux), Python is often the language of choice for hobbyist and experimental userspace applications. As such there are several Python libraries available for interacting

with 1-Wire devices via the Linux sysfs interface. The de facto standard Python library packaged with Raspberry Pi OS, ‘w1thermsensor‘, reads sensors one at a time in released versions [10]. This results in a cost of approximately 750 ms (12-bit conversion) per sensor [2], so N sensors cost approximately N conversion times. This notably ignores the update to the linux kernel which supports bulk reading [3]. Users have requested bulk read functionality (w1thermsensor issue #115), and PR #120 implements it but remains unmerged [11]. Other less prolific Python modules do utilize this kernel feature, but none of them prevent stale reads, which can return trigger-time values hours later with no expiry [12]. This is a hazard for monitoring and control loops, as it can result in silent stale or wrong values, which are worse than errors. The existing bulk consumers (w1-therm-api [13], wb-mqtt-w1 [14]) do not prevent this at the API level. This is a significant gap because multi-sensor setups are the main draw of the 1-Wire protocol, and they scale linearly in read latency under sequential reads. Additionally, generic failure values (w1-therm-api returns ‘None‘ for every failure) hide the cause of the failure, making it difficult to diagnose issues such as wiring problems, CRC errors, power-on resets, or missing drivers.

C. Scope

This project is limited in scope by the following hardware and software constraints:

- **Hardware:** Raspberry Pi 5, DS18B20 only [2], 10 sensors on one bus, externally powered (not parasite)
- **Software:** Linux kernel w1 sysfs interface [3]; no userspace 1-Wire timing
- **Kernels tested:** ‘6.18.50+rpt-rpi-2712‘ (Raspberry Pi OS, Debian 13 trixie) ‘[will verify at time of experiment execution. Stated as the current version]‘; second distribution ‘[TBD]‘ — third distribution ‘[TBD]‘

It provides exactly the following features:

1) **rs1wire:** Rust crate for the kernel ‘w1_therm‘ sysfs interface with discovery, individual reads, bulk reads, resolution/conv_time configuration
2) **Bulk-read API:** that refuses stale reads by construction (double read, expired window, sensor not in conversion, pending result before an individual read)
3) **Typed error taxonomy:** with a distinct variant per failure cause; parser never panics
4) **Independent CRC-8 recomputation:** cross-checked against the kernel’s verdict on every read path
5) **Reproducible benchmark:** that separates the architecture effect (bulk vs. sequential) from the language/library effect (Rust vs. Python if the next feature is implemented)
6) **(Stretch goal) PyO3 Python bindings:** with the global interpreter lock (GIL) released during blocking reads ‘[only if built]‘

II. Related Work

As discussed previously, there are existing software packages written in multiple programming languages which target a similar problem space to rs1wire. The following subsections summarize the relevant work, organized by programming language and approach which illustrates the gap rs1wire aims to fill.

A. Python

w1thermsensor (Timo Furrer; PyPI 2.3.0, Sep 27 2023) [10]

- De facto standard; packaged in Raspberry Pi OS [VERIFY - cite it]
- Reads ‘w1_slave‘ per sensor; independent blocking file I/O
- No ‘therm_bulk_read‘ in any released version (verified by grepping tagged releases still need to cite version tags checked)
- Documented multi-sensor pattern is a sequential loop of ‘get_temperature()‘
- CRC checking implemented correctly; the gap is the missing bulk path, not carelessness
- Ships ‘AsyncW1ThermSensor‘; concurrency does not help on one bus (see Background, bus mutex)

w1thermsensor PR #120, “Add bulk read temperature method” (opened Feb 13 2025, unmerged) [11]

- Closes issue #115 (users requesting bulk read): evidence of demand
- Author-reported: 5 sensors, 12-bit, 4.02 s to 0.90 s; a user reported 7 sensors 4.8 s to 0.8 s (Feb 2026)
- Informal, unreviewed numbers: use as sanity check on expected effect size, not as a result

w1-therm-api (troxel; PyPI) [13]

- Genuine bulk read utilizing Linux sysfs: ‘start_conversion()‘ writes ‘trigger‘, ‘wait_for_conversion()‘ polls every 5 ms, ‘read_temperatures()‘ reads each ‘temperature‘ file
- No freshness enforcement:
  - ‘read_temperatures()‘ without a trigger silently does sequential per-sensor conversions
  - Reading long after a trigger silently returns trigger-time values
  - Partial reads then ‘read_temperatures()‘ silently mix stale bulk and fresh values
- Every failure (missing sensor, CRC, unreadable file, power-on value) returns ‘None‘; trigger failures only printed
- Falls back to ‘w1_slave‘ on invalid/‘85000‘; per kernel source that second access starts a fresh conversion, silently adding one conversion time [VERIFY - grab kernel source w1_therm.c line numbers to cite]
- Treats every 85000 as power-on reset (discards genuine 85 C)
- Small/new: 3 commits, 0 stars

Python performance in systems/hardware code (motivation for a compiled core)

- Rust/C/C++ comparably fast, generally outperform Python [15]
- On ESP32, MicroPython is orders of magnitude slower than compiled languages (e.g. CRC-32 986 us vs 3 us C vs 6.6 us Rust) [16]
- Pure-Python reimplementations of C-backed routines: 6x to 17,000x slower [17]
- For this workload the 750 ms conversion dominates [2]. Language speed only matters in per-sensor overhead

B. Rust

embedded-hal 1-Wire crates (fuchsnj/one-wire-bus, fuchsnj/ds18b20, others on crates.io 'onewire' keyword) [5], [6], [18]

- Designed for bare-metal MCUs; implement 1-Wire timing by toggling GPIO with delays
- Assume no OS preemption mid-pulse; that assumption breaks in Linux userspace [8]
- Rust embedded community frames 1-Wire as a bit-banging HAL problem (embedded-hal issue #54) [19]

w1_therm_reader (gaetronik, v0.1.0, Sep 1 2019) [9]

- Only Rust crate found that reads kernel 'w1_slave'
- 'temp.parse().unwrap()': malformed 't=' line panics despite 'io::Result' return type
- CRC check is only the kernel's 'crc=YES' string; no recompute
- README example references a nonexistent function
- No discovery, bulk read, resolution/conv_time, 85 C handling; sysfs root hardcoded
- Built on nom 5; unchanged since release, 0 stars

Rust in embedded generally: value and open problems [4]; ecosystem overview [20]

C. Other languages / applications

wb-mqtt-w1 (Wiren Board, C++) [14]

- Detects 'therm_bulk_read' per bus master, uses bulk when supported
- Application daemon (MQTT), not a reusable library

D. Python bindings for Rust (only if stretch goal is built)

- PyO3 gives real speedups but leaves per-call FFI overhead (0.14 ms/call plus constant base) [21]
- Speedup contingent on workload shape [22]

E. Gap summary

Table II compares the libraries above against rs1wire on the features this work targets.

III. Background

A. 1-Wire protocol

The 1-wire protocol is a simple, low-cost, single-master, multi-slave communication system that can use a single data line for both power and data transmission [1]. Devices can be powered externally (approach used in this work) or by parasite power from the data line. The master initiates all communication on the bus and controls the timing of all transactions. A transaction begins with a reset pulse from the master, followed by a presence pulse from the slave(s). Bits are sent in time slots on the order of tens of microseconds, with the master and slaves sampling at fixed offsets meaning that timing errors can easily corrupt bits. Each 1-Wire device has a unique 64-bit Read Only Memory (ROM) ID comprised of an 8 bit family code, a 48-bit serial number, and an 8-bit CRC [1]. Software (bit-banged) masters are common, but the precise timing of the protocol is difficult to perform correctly in spaces where the software isn't in total control of the hardware [23].

B. DS18B20



Fig. 1. DS18B20 waterproof probe. Image from [24].

- Family code '0x28' [2]
- Range -55 to +125 C; +/-0.5 C accuracy from -10 to +85 C [2]
- Configurable resolution 9-12 bits; max conversion time 93.75 / 187.5 / 375 / 750 ms [2]
- Convert T command starts conversion; result read from 9-byte scratchpad [2]
- Scratchpad byte 8 is Dallas/Maxim CRC-8 over bytes 0-7 (polynomial $x^8 + x^5 + x^4 + 1$) [2]
- Power-on reset value of the temperature register is +85 C [2]; indistinguishable from a genuine 85 C reading
- Resolution lives in scratchpad RAM (lost on power off unless copied to EEPROM) [2]
- Convert T can be addressed to all devices at once (Skip ROM), which is what makes bulk conversion possible [2]

C. Linux w1 subsystem

- 'w1-gpio': bus master driver that bit-bangs a GPIO pin from kernel space
  - Enabled on Raspberry Pi via 'dtoverlay=w1-gpio' in '/boot/firmware/config.txt' (default GPIO4;

TABLE II
GAP SUMMARY

Library	Lang	Kernel sysfs	Discovery	Bulk	Stale-safe	Typed errors	Config	Maintained
w1thermsensor 2.3.0	Py	yes	yes	no (PR #120 open)	inherent (no bulk)	yes, partial	yes, partial	yes
w1-therm-api	Py	yes	yes	yes	no	no (‘None‘)	no	new
wb-mqtt-w1	C++	yes	yes	yes	yes, partial	n/a	no	yes
one-wire-bus / ds18b20	Rust	no (bit-bang)	yes	yes (Skip ROM)	no (caller-managed)	yes	yes	no
w1_therm_reader	Rust	yes	no	no	inherent (no bulk)	no (io::Error + string)	no	dormant (2026 dependency bump only)
rs1wire	Rust	yes	yes	yes	**yes**	**yes**	yes	yes

‘gpiopin=‘ to change) ‘[VERIFY + cite RPi config docs]‘

- ‘w1_therm‘: slave driver for temperature sensors [3]
- sysfs layout:
  - ‘/sys/bus/w1/devices/w1_bus_masterN/‘ with ‘w1_master_slaves‘, ‘w1_master_slave_count‘, ‘w1_master_search‘, ‘therm_bulk_read‘
  - ‘/sys/bus/w1/devices/28-xxxxxxxxxxxx/‘ (symlink into its bus master) with ‘w1_slave‘, ‘temperature‘, ‘resolution‘, ‘conv_time‘, others [12]
- ‘w1_slave‘ format (two lines):
  - Line 1: 9 scratchpad bytes, ‘: crc=XX YES|NO‘ [3]
  - Line 2: bytes of the last scratchpad that passed CRC (not necessarily the current one), then ‘t=<millidegrees C>‘
- Reading ‘w1_slave‘ or ‘temperature‘ with no pending bulk result triggers a single-sensor conversion and blocks conv_time [12]
- ‘therm_bulk_read‘ (per bus master):
  - Write ‘trigger‘: Convert T to all sensors on the bus [3]
  - Write is synchronous: kernel sleeps for the longest ‘conv_time‘ on the bus before returning (from ‘w1_therm.c‘, rpi-6.18.y) ‘[VERIFY line refs]‘
  - Read: ‘-1‘ conversion in progress, ‘1‘ complete with unread results, ‘0‘ nothing pending [3], [12]
  - Default permissions 0644 root: unprivileged users cannot trigger
- Stale-read mechanism (kernel source, rpi-6.18.y):
  - Each sensor has a pending-result flag set by the bulk trigger
  - ‘w1_slave_show‘ / ‘temperature_show‘: if flag is 1, read scratchpad and clear it; if 0, call ‘convert_t()‘ (fresh conversion)
  - No expiry: a result read hours later is still the trigger-time value
  - Consequence 1: late read returns stale data silently [12]
  - Consequence 2: second read of same sensor silently costs a new full conversion
- Read failures: kernel returns an empty file rather than an error code
- Bus mutex (rpi-6.18.y): without strong pull-up (externally powered), held through the conversion sleep, so concurrent per-sensor conversions on one bus serialize; with strong pull-up it is released before sleeping ‘[VERIFY on hardware: async vs. sequential timing]‘
  - Implication: threads/asyncio cannot recover the bulk advantage on a single externally powered bus

D. Userspace timing on Linux (why not bit-bang in userspace)

- Userspace worst-case GPIO latency 147-160 us vs. kernel-space 67-76 us on Pi 3 / BeagleBone [8] ‘[VERIFY figures]‘
- PREEMPT_RT brings jitter below 50 us [25]
- Even RT Linux shows tens of us jitter; sub-us needs interrupt isolation [26] (preprint)
- Real-time tuning needs isolcpus, IRQ affinity, tickless operation [27]
- 1-Wire slots are tens of us [23], so these latencies are the same order as the slot timing
- Caveat to state: kernel ‘w1-gpio‘ also bit-bangs; kernel space reduces but does not eliminate the problem, and rs1wire inherits it

E. Raspberry Pi 5 / RP1 specifics (primary sources only, say so)

- GPIO moved off the SoC to the RP1 I/O controller over PCIe; inherently higher latency than on-die GPIO [28]
- Most PIOLib operations take at least 10 us [29]
- Direct register access libraries (pigpio, RPi.GPIO) broke on Pi 5 [30]
- RP1 registers proxied via firmware mailbox in some kernel paths [31] ‘[VERIFY context]‘
- Earlier w1-gpio instability on Pi 5 fixed from kernel 6.6.28+ ‘[VERIFY: replace forum source with kernel commit/changelog]‘
- ‘[VERIFY]‘ 1 us direct-write figure from RP1 datasheet before quoting
- Open note: Raspberry Pi docs inconsistent on PCIe generation (Gen 2 x4 vs. Gen 3); avoid stating it or cite carefully

IV. Methodology

A. Design goals

The rs1wire library is designed to consume only kernel-produced results so as to avoid implementing 1-Wire timing in userspace and inheriting the associated latency and jitter issues. It also makes stale bulk reads impossible to do by accident via runtime checks and type system enforcement. It supports user's understanding of potential issues with one error variant per failure case and by never panicking on kernel output.

B. API structure (Rust)

The public API mirrors the sysfs hierarchy in three handle types, and every fallible call returns a typed 'Error' rather than a sentinel value. The entry point, 'W1', holds only the sysfs root path. It defaults to '/sys/bus/w1/devices', but 'W1::with_root()' accepts any directory, so tests can run against fixture trees instead of hardware. From it, 'bus_masters()' and 'bus(name)' return bus handles, and 'sensors()' and 'sensor(id)' find DS18B20s across all buses. A missing root, or one with no bus masters, is reported as the 'w1-gpio' module not being loaded.

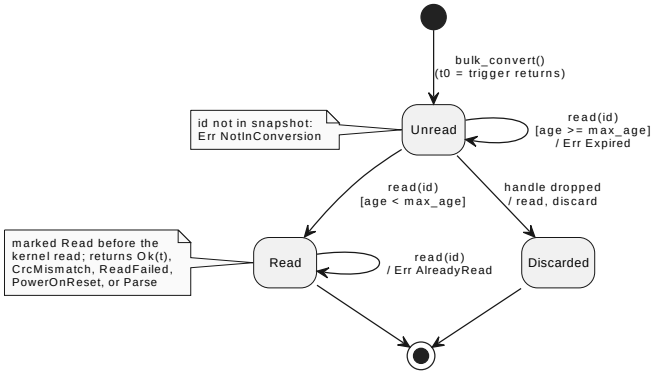
A 'BusMaster' discovers its sensors from the kernel's 'w1_master_slaves' list, keeping only family '0x28'. It also owns the bulk path: 'bulk_status()' maps the 'therm_bulk_read' values '-1', '1', and '0' to an enum, and 'bulk_convert()' writes 'trigger' and returns a 'BulkConversion' handle (Section IV-C). 'discard_pending()' clears results left behind by another process.

A 'Sensor' performs individual reads through 'read()', or 'read_with()' to change the 85 °C policy, and refuses to read while a bulk result is pending. It also gets and sets 'resolution' and 'conv_time'. These writes require root, and the resolution lives in sensor RAM, since saving it to EEPROM is not supported; changing it also resets any 'conv_time' override.

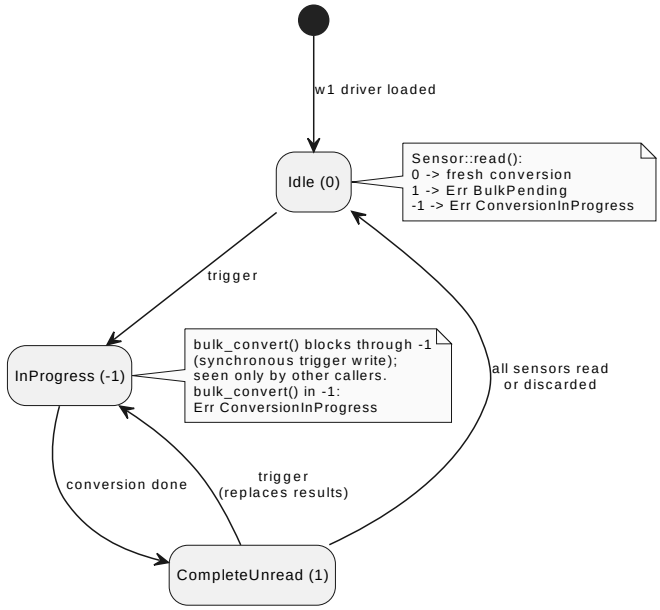
The value types encode the kernel's rules so that invalid input cannot be expressed. 'DeviceId' is parsed strictly from the kernel's '28-xxxxxxxxxxxx' name, and 'Temperature' stores integer millidegrees exactly as reported, with no floating-point rounding until the caller asks for degrees. 'Resolution' is a four-variant enum covering 9 to 12 bits. 'ConvTime' is 'DEFAULT', 'MEASURE' (the kernel times a real conversion), or a fixed number of milliseconds of at least 2, since the kernel reserves 0 and 1 for the first two.

C. Stale-read prevention (core contribution; candidate figure: state diagram)

The kernel's bulk-read interface leaves freshness entirely to the caller. After a 'trigger', each sensor holds a pending result that is returned by its next read, whenever that read happens, and is then cleared [12]. Two hazards follow. A read made long after the trigger silently returns the trigger-time value, and a second read of the same sensor silently starts a fresh single-sensor conversion, costing a full conversion time. rs1wire addresses both by tying every bulk result to the one conversion that produced it. 'bulk_convert()' does not return temperatures directly; it returns a 'BulkConversion' handle, and the only way to obtain a bulk result is through that handle. Fig. 2a traces one sensor's result through that handle, and Fig. 2b shows the bus-wide status the kernel reports alongside it.



(a) Per-sensor states within one 'BulkConversion' handle



(b) Bus status reported by 'therm_bulk_read'

Fig. 2. Bulk-read state machines. Refusals in (a) are enforced per handle within one process.

Wherever possible, the handle's guarantees are enforced at compile time. The 'BulkConversion' mutably borrows its 'BusMaster', so Rust's borrow checker allows at most one live conversion per bus handle, and a new trigger cannot be issued on that handle while results from the previous one are still outstanding. 'read_all()' takes the handle by value, consuming it, so reading the whole conversion twice is a compile error rather than a runtime bug; a 'compile_fail' doctest checks that this code is rejected.

The remaining hazards depend on which sensor is requested and on elapsed time, so they cannot be expressed in types and are instead checked at runtime when 'read(id)' is called. Each refusal is an 'Error::StaleBulkRead' carrying a 'StaleReason'

that names the specific hazard. ‘AlreadyRead‘ rejects a second read of a sensor, which the kernel would otherwise serve with a new conversion. ‘Expired‘ rejects any read made later than ‘max_age‘ after the conversion completed. Because the kernel has no expiry of its own, this window is a policy of the crate rather than a physical limit; it defaults to 1 s and is measured from the moment the ‘trigger‘ write returns, which is when the kernel marks results ready. ‘NotInConversion‘ rejects a sensor that was not on the bus when the conversion began, using the sensor list taken from ‘w1_master_slaves‘ just before the trigger.

Results that are never requested also need handling, because the kernel would otherwise keep them and hand them to some later, unrelated read. When a ‘BulkConversion‘ is dropped, it reads and discards every result it did not hand out, at roughly 10–15 ms per sensor; sensors refused as ‘Expired‘ stay on this list, so their stale values are cleared as well. The same concern applies to individual reads made outside any handle. ‘Sensor::read()‘ first checks the bus’s ‘therm_bulk_read‘ status and refuses with ‘BulkPending‘ if unread bulk results exist, since the kernel would return them in place of a new conversion. Results left by another process can be cleared with ‘BusMaster::discard_pending()‘. Likewise, ‘bulk_convert()‘ refuses with ‘ConversionInProgress‘ if a conversion is already running on the bus.

These guarantees are complete only within a single process. The status checks read ‘therm_bulk_read‘ and then act on it, so another process that triggers a conversion between the check and the read can still cause a stale or unexpected result. This race is a limitation of the sysfs interface, which offers no locking, and it is documented in the API.

D. Integrity checks

Every temperature rs1wire returns, from individual and bulk reads alike, comes from the sensor’s ‘w1_slave‘ file. Unlike ‘temperature‘, which holds only the final value, ‘w1_slave‘ includes the nine raw scratchpad bytes and the kernel’s CRC verdict, which makes the following checks possible. w1-therm-api, by contrast, reads bulk results from ‘temperature‘ and has nothing to check them against.

When the kernel’s own read fails (a bus error, a CRC failure while fetching a bulk result, or an unresponsive sensor), it returns an empty file rather than an error code. rs1wire reports this as ‘ReadFailed‘, or as ‘ConversionInProgress‘ if another process is mid-conversion. Non-empty content is parsed strictly, and anything unexpected is a ‘Parse‘ error rather than a panic. The second line’s bytes belong to the last scratchpad that passed CRC, not necessarily the current one, so they are checked only for shape.

rs1wire then recomputes the Dallas/Maxim CRC-8 over bytes 0–7 instead of trusting the kernel. A mismatch with byte 8, a kernel ‘NO‘, or a kernel CRC that differs from the recomputed one is reported as ‘CrcMismatch‘, so the two calculations can never disagree silently. An all-zero scratchpad passes CRC-8 but means nothing drove the bus, so it is caught first as ‘ReadFailed‘. The reported temperature is the kernel’s ‘t=‘ conversion of these CRC-checked bytes; rs1wire verifies the bytes but does not decode the temperature itself.

Finally, the DS18B20 reads 85 °C after power-on until its first conversion completes [2], so an 85 °C reading usually means lost power or an early read, yet is indistinguishable from a genuine one. By default (‘PowerOnResetPolicy::Reject‘) rs1wire returns ‘PowerOnReset‘, and its documentation states that this discards genuine 85 °C readings; applications that need them can opt in with ‘PowerOnResetPolicy::Accept‘. w1-therm-api makes the same trade-off silently, returning ‘None‘.

E. Error taxonomy

Table III lists every error variant, the condition that produces it, and how the caller should respond.

TABLE III
ERROR TAXONOMY

Variant	Cause / signal	Caller action
Setup and discovery		
‘ModuleNotLoaded‘	–	–
‘BusNotFound‘	–	–
‘DeviceNotFound‘	–	–
‘BulkReadUnsupported‘	–	–
Reading and data integrity		
‘ReadFailed‘	–	–
‘CrcMismatch‘	–	–
‘PowerOnReset‘	–	–
‘Parse‘	–	–
Bulk-read coordination		
‘ConversionInProgress‘	–	–
‘StaleBulkRead‘:	–	–
‘AlreadyRead‘	–	–
‘Expired‘	–	–
‘NotInConversion‘	–	–
‘BulkPending‘	–	–
Other		
‘Io‘	–	–

F. Parser

All parsing of kernel output lives in pure functions with no I/O, which keeps it independently testable. The parser accepts only the exact format the kernel prints: two lines, exactly nine two-digit hex bytes, a ‘YES‘ or ‘NO‘ verdict, and plain decimal integers. Anything else, including ‘+‘ signs, overflowing values, extra lines, or wrong byte counts, is returned as a ‘Parse‘ error carrying a truncated copy of the offending input, so malformed sysfs content can never cause a panic.

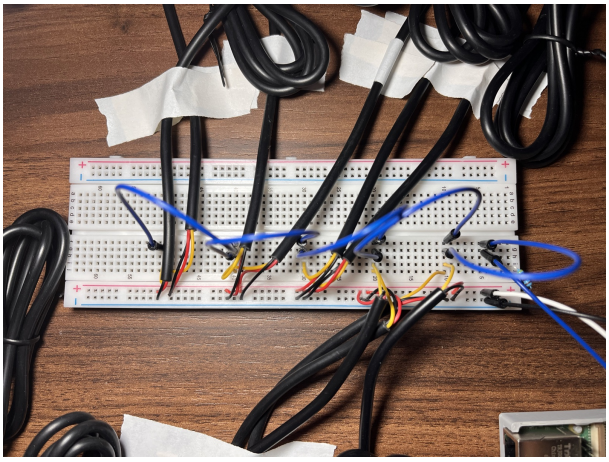
G. Correctness testing

Because the sysfs root is configurable, the 46 unit tests run against fixture directory trees instead of hardware. They cover discovery across multiple buses, empty buses, and foreign device families; every CRC and read-failure path; the power-on reset policy; each bulk-read refusal; and configuration reads and writes. A ‘compile_fail‘ doctest confirms that a consumed ‘BulkConversion‘ cannot be read again. Fixtures are plain files, however, so they cannot model the kernel clearing a bulk result

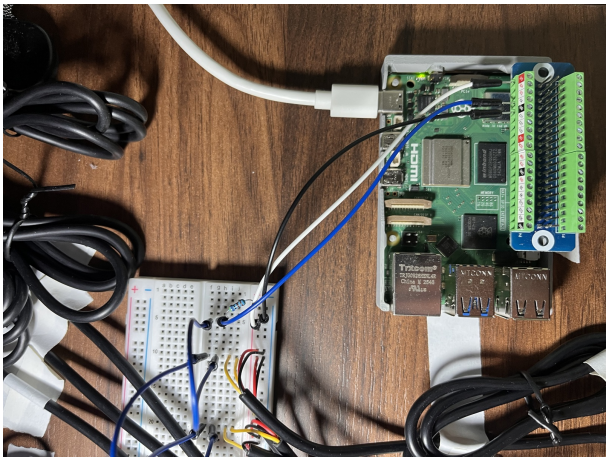
when it is read. That behavior is checked on hardware by ‘examples/smoke.rs‘, and the error rate of every method is also recorded during benchmark runs.

H. Hardware and software environments

All measurements use a Raspberry Pi 5 (1 GB) with ten DS18B20 waterproof probes on a single ‘w1-gpio‘ bus (Figs. 3 and 4). The sensors are externally powered rather than parasite-powered and share one 4.7 kΩ pull-up resistor. The bus is enabled with the ‘w1-gpio‘ device tree overlay in ‘/boot/firmware/config.txt‘ on GPIO pin 4, and sensors run at 12-bit resolution with the kernel’s default ‘conv_time‘.



(a) Breadboard



(b) Raspberry Pi 5

Fig. 3. Test hardware: ten DS18B20 sensors on one w1-gpio bus.

The primary software environment is Raspberry Pi OS (Debian 13 “trixie”) with kernel ‘6.18.50+rpt-rpi-2712‘; a second and third distribution tested on the same board are ‘[TBD]‘ and ‘[TBD]‘. The Python baselines are w1thermsensor 2.3.0 and w1-therm-api 0.1.0 under Python 3.13.5, and rs1wire is a release build from rustc 1.99.0 (b940084d7 2026-09-28). Since ‘therm_bulk_read‘ is writable only by root by default, a udev rule grants the benchmark user’s group write access so that bulk methods run unprivileged. Every row of the benchmark output records the kernel, OS, library versions and source commit, resolution, and ‘conv_time‘, so each result can be traced to its exact environment.

I. Experimental design

Table IV lists the benchmarked methods. Each library is run both sequentially and, where it supports it, in bulk. The same CSV file is written to for all methods to ensure that results are interpretable regardless of the tool or language used.

TABLE IV
BENCHMARK METHODS

Method	Reads	Calls per cycle
‘w1thermsensor-seq‘ (Py)	‘w1_slave‘	‘get_temperature()‘ on each sensor
‘w1-therm-api-seq‘ (Py)	‘temperature‘	‘read_temperatures()‘ with no trigger
‘w1-therm-api-bulk‘ (Py)	‘temperature‘	‘start_conversion()‘, ‘wait_for_conversion()‘, ‘read_temperatures()‘
‘rs1wire-seq‘ (Rust)	‘w1_slave‘	‘Sensor::read()‘ on each sensor
‘rs1wire-bulk‘ (Rust)	‘w1_slave‘	‘bulk_convert()‘, then ‘BulkConversion::read()‘ on each sensor

The methods are chosen in pairs so that each comparison changes as little as possible. Comparing w1-therm-api’s sequential and bulk methods isolates the effect of bulk conversion within a single codebase, while comparing w1thermsensor’s sequential reads against either bulk method shows the same effect against the de facto standard library. With the architecture held fixed, w1-therm-api-bulk against rs1wire-bulk isolates the language and library effect, and the three sequential methods compare per-call overhead across libraries when every read pays a full conversion. If the optional asynchronous w1thermsensor method is run, comparing it with bulk reads tests whether concurrency can close the gap; the bus mutex described in Section III predicts that it cannot on an externally powered bus.

Each method is measured at fixed sensor counts of $N = 1, 2, 5$, and 10, using the first N sensors by ROM ID in both harnesses. Within each trial, every (method, N) cell runs in a seeded random order, and the seed and actual run order are recorded so that slow drift is spread across methods rather than aligned with one. One unrecorded warmup trial precedes the recorded ones, and ‘run_all.sh‘ alternates harness blocks in the order Python, Rust, Rust, Python, with 15 trials per block (30 per library). Since a bulk trigger converts every sensor on the bus, any results left unread after a bulk cell are drained untimed, and ‘therm_bulk_read‘ is checked before every cell, with any leftover result flagged in the output. With the full bus present, the kernel’s periodic bus search is disabled for the run and restored on exit, so a search cycle cannot hold the bus mutex mid-trial.

The following metrics are collected during each trial run:

- **Wall-clock time:** measured with ‘perf_counter‘ in Python and ‘Instant‘ in Rust.

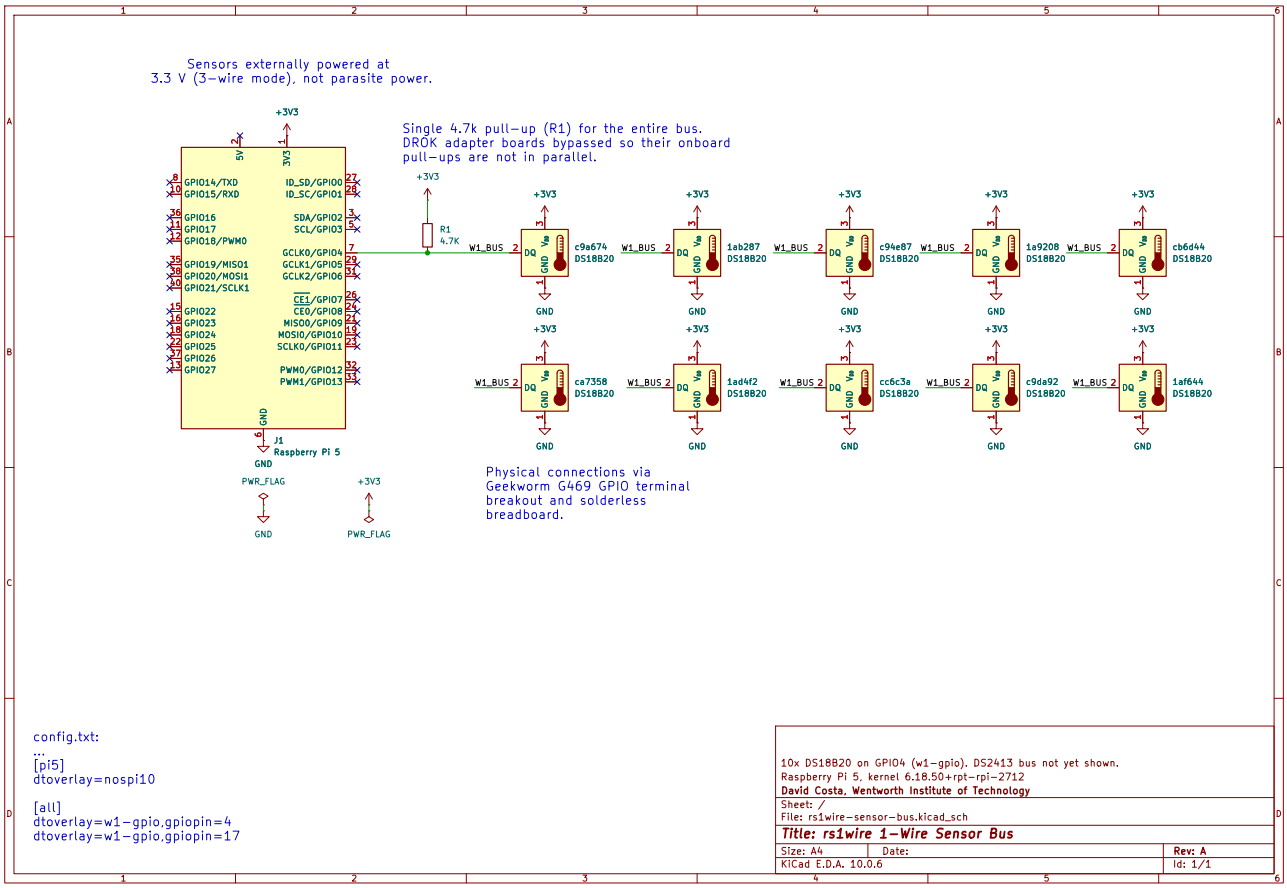


Fig. 4. Schematic of the 1-Wire sensor bus.

- **CPU time:** total process CPU time from ‘process_time‘ (Python) and ‘CLOCK_PROCESS_CPUTIME_ID‘ (Rust), split into user and system time with ‘getrusage‘. The split is tick-sampled, so it is approximate. w1-therm-api’s 5 ms polling loop counts toward its CPU time.
- **Phase timings:** for bulk methods, the trigger, wait, and readout phases are timed separately where possible. Because the kernel’s trigger write is synchronous, w1-therm-api’s trigger phase contains the whole conversion and its wait phase is near zero; rs1wire has no separate wait phase and leaves it empty.
- **Read outcomes:** the number of successful and failed reads, and the error type of each failure.

J. Expected outcomes and threats to validity

The outcome for each trial has not been confirmed yet but is expected to show parity across all libraries at N=1; show rs1wire clearly faster than w1thermsensor-seq at $N > 1$ and at least competitive with w1-therm-api-bulk. w1thermsensor PR #120 shows 4.5x speedup from sequential to bulk for 5 sensors [11] and will be used as a sanity check for rs1wire.

A handful of potential threats to the experiment’s validity have been identified prior to execution; Table V lists each one and how it is addressed.

TABLE V
THREATS TO VALIDITY

Threat	Mitigation
Rust and Python harnesses run as separate processes, so their cells are not interleaved in one shuffle	Blocks run in py, rs, rs, py order so slow drift does not align with one language
Single board, single bus, at most 10 sensors; results may not generalize to larger buses or other masters (e.g. DS2482 I^2C bridges)	None; claims are limited to the tested configuration
Bulk semantics and bus mutex behavior depend on the kernel version	Kernel version recorded with every result; behavior checked against the ‘rpi-6.18.y‘ source
Room temperature drifts across runs	Seeded random cell order and alternating blocks spread drift across methods

V. Proposed System & Architecture

VI. Results

A. Figures and Tables

a) Positioning Figures and Tables: Place figures and tables at the top and bottom of columns. Avoid placing them in the middle of columns. Large figures and tables may span across both columns. Figure captions should be below the figures; table heads should appear above the tables. Insert figures and tables after they are cited in the text. Use the abbreviation "Fig. 5", even at the beginning of a sentence.

Tables VI and VII summarize each (method, N) cell over all recorded trials. Wall-clock time is reported as the median with the interquartile range in brackets; the other timings are medians. For bulk methods, Trig. and Read are the trigger and readout phases. Failed reads are totals across all trials.

TABLE VI
Sequential Polling Results

N	**Wall (s)**	**CPU (s)**	**Failed reads**
'w1thermsensor-seq'			
1	–	–	–
2	–	–	–
5	–	–	–
10	–	–	–
'w1-therm-api-seq'			
1	–	–	–
2	–	–	–
5	–	–	–
10	–	–	–
'rs1wire-seq'			
1	–	–	–
2	–	–	–
5	–	–	–
10	–	–	–

TABLE VII
Bulk Polling Results

N	**Wall (s)**	**Trig. (s)**	**Read (s)**	**CPU (s)**	**Failed**
'w1-therm-api-bulk'					
1	–	–	–	–	–
2	–	–	–	–	–
5	–	–	–	–	–
10	–	–	–	–	–
'rs1wire-bulk'					
1	–	–	–	–	–
2	–	–	–	–	–
5	–	–	–	–	–
10	–	–	–	–	–

Fig. 5. Example of a figure caption.

Figure Labels: Use 8 point Times New Roman for Figure labels. Use words rather than symbols or abbreviations when writing Figure axis labels to avoid confusing the reader. As an example, write the quantity "Magnetization", or "Magnetization, M", not just "M". If including units in the label, present them within parentheses. Do not label axes only with units. In the example, write "Magnetization (A/m)" or "Magnetization {A[m(1)]}", not just "A/m". Do not label axes with a ratio of quantities and units. For example, write "Temperature (K)", not "Temperature/K".

VII. Discussion

VIII. Conclusion

Be sure that the symbols in your equation have been defined before or immediately following the equation. Use "(**??**)", not "Eq. (**??**)" or "equation (**??**)", except at the beginning of a sentence: "Equation (**??**) is . . ."

Acknowledgment

The author would like to thank all contributors and reviewers for their valuable feedback and support. Might acknowledge Professor Ashar here? Not sure if he gets his own spot in the author's list or just an acknowledgment as an advisor.

References

[1] S. Larsen, "Guide to 1-Wire communication," June 19 2008, accessed: Oct. 14, 2026. [Online]. Available: https://www.analog.com/en/resources/technical-articles/guide-to-1wire-communication.html

[2] Analog Devices, "DS18B20 digital thermometer datasheet," 2019, accessed: 10/07/2026. [Online]. Available: https://www.analog.com/media/en/technical-documentation/data-sheets/DS18B20.pdf

[3] Linux Kernel Documentation, "Kernel driver w1_therm," March 2023, accessed: 10/08/2026. [Online]. Available: https://docs.kernel.org/w1/slaves/w1_therm.html

[4] A. Sharma, S. Sharma, S. R. Tanksalkar, S. Torres-Arias, and A. Machiry, "Rust for embedded systems: Current state and open problems," in *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS '24)*. Salt Lake City, UT, USA: ACM, Oct. 2024, pp. 2296–2310.

[5] fuchsnj, "one-wire-bus," January 2020, version 0.1.1. Accessed: 09/22/2026. [Online]. Available: https://github.com/fuchsnj/one-wire-bus

[6] ——, "ds18b20," August 2020, version 0.1.1. Accessed: 09/22/2026. [Online]. Available: https://github.com/fuchsnj/ds18b20

[7] D. Molloy, "The state of rust for embedded development in mid-2026," July 2026, accessed: 10/29/2026. [Online]. Available: https://derekmolloy.ie/the-state-of-rust-for-embedded-development-in-mid-2026/

[8] G. K. Adam, N. Petrellis, and L. T. Doulos, "Performance assessment of Linux kernels with PREEMPT_RT on ARM-based embedded devices," *Electronics*, vol. 10, no. 11, p. 1331, 2021.

[9] Gaetronik, "w1_therm_reader," version 0.1.1. Accessed: 09/19/2026. [Online]. Available: https://github.com/gaetronik/w1_therm_reader

[10] T. O. Furrer, "w1thermsensor," version 2.3.0. Accessed: 09/19/2026. [Online]. Available: https://github.com/timofurrer/w1thermsensor

[11] SimonInOps, "w1thermsensor PR 120," February 2025, accessed: 09/22/2026. [Online]. Available: https://github.com/timofurrer/w1thermsensor/pull/120

[12] E. Polyakov, "Kernel driver w1_therm," accessed: 09/20/2026. [Online]. Available: https://www.kernel.org/doc/Documentation/ABI/testing/sysfs-driver-w1_therm

[13] Troxel, "w1-therm-api," version 0.1.0. Accessed: 09/19/2026. [Online]. Available: https://github.com/troxel/w1-therm-api

[14] "wb-mqtt-w1," version 2.4.3. Accessed: 10/29/2026. [Online]. Available: https://github.com/wirenboard/wb-mqtt-w1

[15] W. Bugden and A. Alahmar, "The safety and performance of prominent programming languages," *International Journal of Software Engineering and Knowledge Engineering*, vol. 32, no. 5, pp. 713–744, 2022.

[16] I. Plauska, A. Liutkevičius, and A. Janavičiūtė, "Performance evaluation of C/C++, MicroPython, Rust and TinyGo programming languages on ESP32 microcontroller," *Electronics*, vol. 12, no. 1, p. 143, 2022.

[17] P. Ding and R. Stevens, "Stdlib or third-party? empirical performance and correctness of LLM-assisted zero-dependency Python libraries," *arXiv preprint arXiv:2605.21405*, 2026.

[18] "Onewire keyword search on crates.io," accessed: 10/29/2026. [Online]. Available: https://crates.io/keywords/onewire

[19] reportingsjr, "1-wire API," March 2018, accessed: 10/05/2026. [Online]. Available: https://github.com/rust-embedded/embedded-hal/issues/54

[20] T. Vandervelden, R. De Smet, D. Deac, K. Steenhaut, and A. Braeken, "Overview of embedded Rust operating systems and frameworks," *Sensors*, vol. 24, no. 17, p. 5818, 2024.

[21] I. B. d. Amaral, R. C. Ferreira, and A. Goldman, "Rust vs. C for Python libraries: Evaluating Rust-compatible bindings toolchains," *arXiv preprint arXiv:2507.00264*, 2025.

[22] K. Harding and D. M. Dunlavy, "Improving runtime performance of tensor computations using Rust from Python," *arXiv preprint arXiv:2510.01495*, 2025.

[23] Analog Devices, "1-Wire communication through software," May 30 2002, accessed: Oct. 7, 2026. [Online]. Available: https://www.analog.com/en/resources/technical-articles/1wire-communication-through-software.html

[24] Voltaat, "Ds18b20 waterproof digital temperature sensor – stainless steel probe," accessed: Oct. 31, 2026. [Online]. Available: https://www.voltaat.com/products/digital-thermometer

[25] G. K. Adam, "Real-time performance and response latency measurements of Linux kernels on single-board computers," *Computers*, vol. 10, no. 5, p. 64, 2021.

[26] Z. Zhou, Z. Liu, S. Zhang, J. Li, D. Du, M. Sun, Z. Wang, H. Liu, and G. Xu, "Towards deterministic sub-0.5 us response on Linux through interrupt isolation," *arXiv preprint arXiv:2509.03855*, 2025.

[27] M. M. Madden, "Challenges using Linux as a real-time operating system," in *AIAA Scitech 2019 Forum*, 2019, p. 0502.

[28] E. Upton, "RP1: the silicon controlling Raspberry Pi 5 I/O, designed here at Raspberry Pi," October 2023, accessed: 09/25/2026. [Online]. Available: https://www.raspberrypi.com/news/rp1-the-silicon-controlling-raspberry-pi-5-i-o-designed-here-at-raspberry-pi/

[29] PhilE, "PIOLib: a userspace library for PIO control," December 2024, accessed: 09/25/2026. [Online]. Available: https://www.raspberrypi.com/news/piolib-a-userspace-library-for-pio-control/

[30] JinShil, "pigpio will not run on a Pi 5," November 2023, accessed: 09/25/2026. [Online]. Available: https://github.com/joan2937/pigpio/issues/589

[31] pelwell, "RP1 PIO support," November 2024, accessed: 10/29/2026. [Online]. Available: https://github.com/raspberrypi/linux/pull/6470

[32] IEEE Computational Intelligence Society, "Manuscript templates for conference proceedings," 2024. [Online]. Available: https://www.ieee.org/conferences/publishing/templates

[33] J. Nanjekye, D. Bremner, and A. Micic, "Towards reliable memory management for Python native extensions," in *Proceedings of the 18th ACM International Workshop on Implementation, Compilation, Optimization of OO Languages, Programs and Systems*, 2023, pp. 15–26.

[34] A. de Faveri Tron, S. Longari, M. Carminati, M. Polino, and S. Zanero, "CANflict: Exploiting peripheral conflicts for data-link layer attacks on automotive networks," in *Proceedings of the 2022 ACM SIGSAC conference on computer and communications security*, 2022, pp. 711–723.

[35] PuddyMuddle, "Temperature reading code not working ?" August 2023, accessed: 10/04/2026. [Online]. Available: https://forums.raspberrypi.com/viewtopic.php?t=354551

[36] J. Pihlajamaa, "Benchmarking Raspberry Pi GPIO speed," July 2012, accessed: 10/20/2026. [Online]. Available: https://codeandlife.com/2012/07/03/benchmarking-raspberry-pi-gpio-speed/

[37] Raspberry Pi Foundation, "RP1 peripherals datasheet," November 2023, accessed: 10/01/2026. [Online]. Available: https://datasheets.raspberrypi.com/rp1/rp1-peripherals.pdf

[38] Adafruit, "Comparing libgpiod and gpiozero speeds on the Raspberry Pi 5," October 2023, accessed: 10/29/2026. [Online]. Available: https://blog.adafruit.com/2023/10/26/comparing-libgpiod-and-gpiozero-speeds-on-the-raspberry-pi-5/